Fintech Project Management Forecasting

Microsoft Elevate AICTE Internship – Capstone Project

Author: Gyanankur Baruah

This notebook demonstrates:

- Importing and cleaning fintech project data
- Exploratory analysis of budgets, risks, and completion rates
- Predictive modeling (ARIMA/LSTM) for project completion and resource demand
- Exporting cleaned dataset for Power BI dashboards

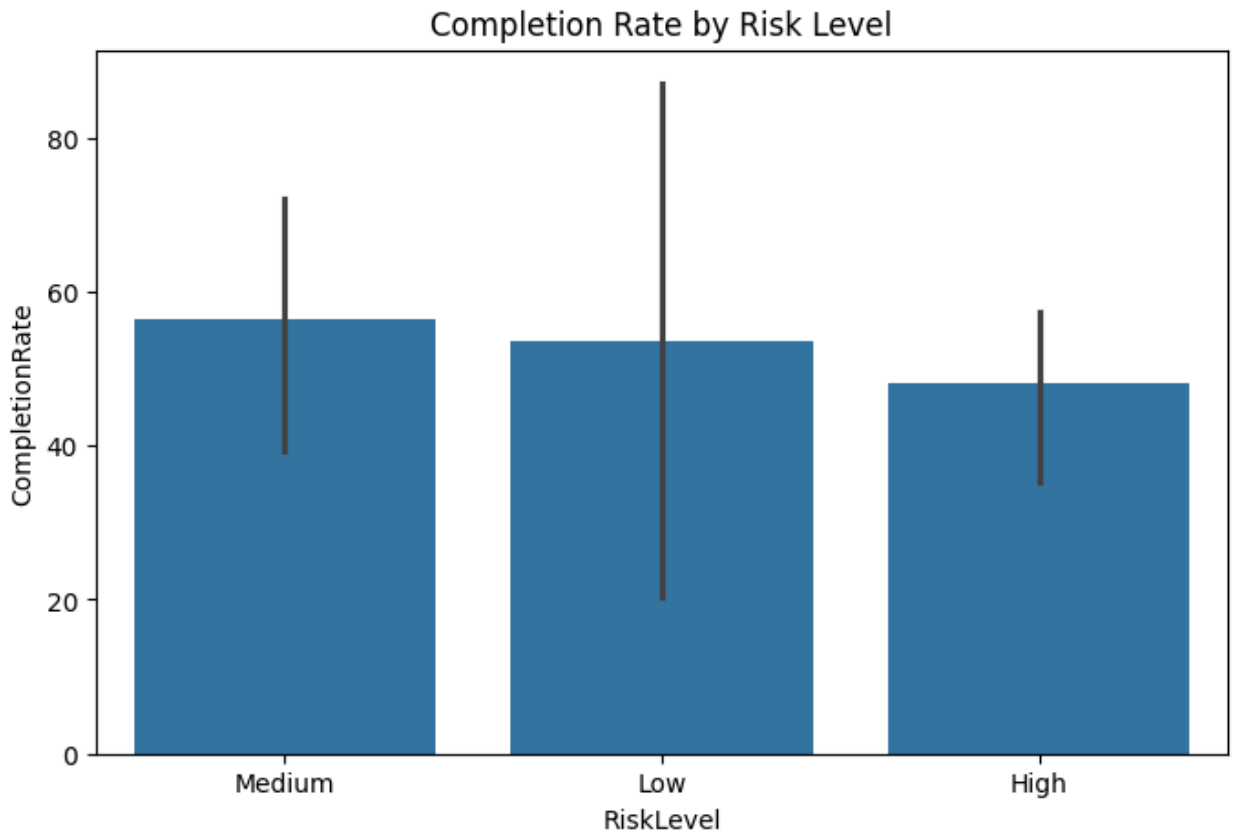
Project Analysis Overview

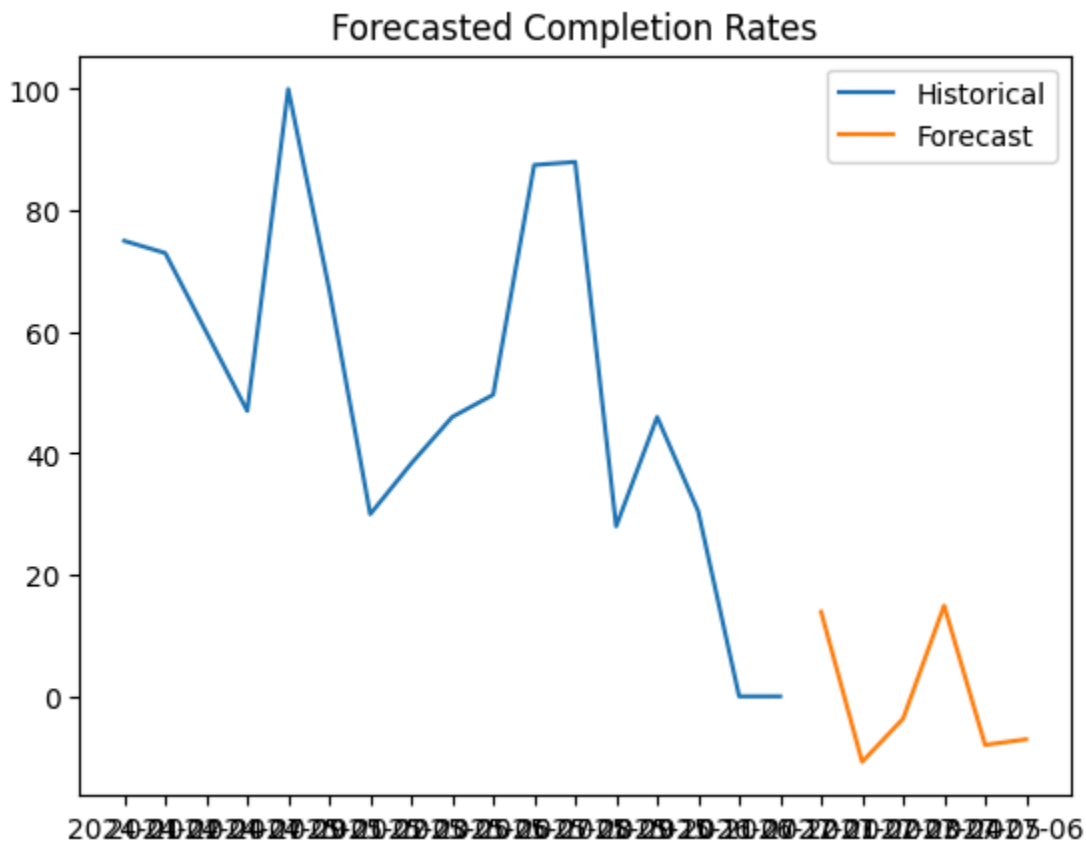
This notebook performs an analysis of Fintech project data, focusing on project completion rates, budget variance, and geographical distribution. It includes data loading, cleaning, exploratory data analysis, and time-series forecasting. The goal is to identify key performance indicators and potential areas for improvement.

In [ ]:
```
# 1. Setup
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
import math

# 2. Load Dataset
file_path = "/content/Fintech_Projects_Challenge32_Dataset.xlsx"
df = pd.read_excel(file_path)

# Preview
print("Dataset Shape:", df.shape)
print("DataFrame columns:", df.columns.tolist()) # Added to inspect columns
df.head()

# 3. Basic Cleaning
# Convert 'PlannedStartDate' to datetime first. Invalid dates will become NaT.
df['PlannedStartDate'] = pd.to_datetime(df['PlannedStartDate'], errors='coerce

# Drop rows where critical columns for time series or budget calculations are
```

```python
# This ensures that we have valid dates and completion rates for forecasting.
critical_columns = ['PlannedStartDate', 'CompletionPercentage', 'ActualBudget
df.dropna(subset=critical_columns, inplace=True)

# Fix: Use 'CompletionPercentage' as 'CompletionRate' and correct budget colum
df['CompletionRate'] = df['CompletionPercentage']
df['BudgetVariance'] = df['ActualBudget (EUR)'] - df['PlannedBudget (EUR)']

# 4. Exploratory Analysis
plt.figure(figsize=(8,5))
sns.barplot(x="RiskLevel", y="CompletionRate", data=df)
plt.title("Completion Rate by Risk Level")
plt.show()

# 5. Forecasting Example (ARIMA)
# 'PlannedStartDate' is already converted and NaNs handled in cleaning.
# Now create 'Month' column.
df['Month'] = df['PlannedStartDate'].dt.to_period('M')

# Group by 'Month'
series = df.groupby("Month")["CompletionRate"].mean()

model = ARIMA(series, order=(2,1,2))
fit = model.fit()
forecast = fit.forecast(steps=6)

# Adjusting x-axis for plotting PeriodIndex objects
plt.plot(series.index.astype(str), series, label="Historical")
# Generate future PeriodIndex for forecast plotting
future_index = pd.period_range(start=series.index[-1] + 1, periods=6, freq='M'
plt.plot(future_index.astype(str), forecast, label="Forecast")
plt.legend()
plt.title("Forecasted Completion Rates")
plt.show()

# 6. Export Cleaned Data for Power BI
df.to_csv("/content/Fintech_Projects_Clean.csv", index=False)
print("Cleaned dataset exported for Power BI dashboard.")
```

```
Dataset Shape: (72, 18)
DataFrame columns: ['ProjectID', 'ProductName', 'DepartmentID', 'ProjectManager
ID', 'Project_Country', 'City', 'Project_Country_Latitude', 'Project_Country_Lo
ngitude', 'PlannedStartDate', 'PlannedEndDate', 'ActualStartDate', 'ActualEndDa
te', 'Status', 'PlannedBudget (EUR)', 'ActualBudget (EUR)', 'RiskLevel', 'Compl
etionPercentage', 'Country_Flag']
```






```
Cleaned dataset exported for Power BI dashboard.
```

Mid-Analysis Insights

Following the initial data cleaning and exploratory analysis, we've observed several key trends regarding project performance across different risk levels, departments, and geographical locations. These insights will help us understand the factors influencing project outcomes and inform our forecasting models.

In [ ]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

values = series.values.reshape(-1,1)
X, y = [], []
for i in range(len(values)-3):
    X.append(values[i:i+3])
    y.append(values[i+3])
X, y = np.array(X), np.array(y)

# Build LSTM model
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(3,1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Train
model.fit(X, y, epochs=50, verbose=0)

# Predict the next single value based on the last sequence in X
pred = model.predict(X[-1].reshape(1,3,1))
print("Next forecasted completion rate:", pred)
```

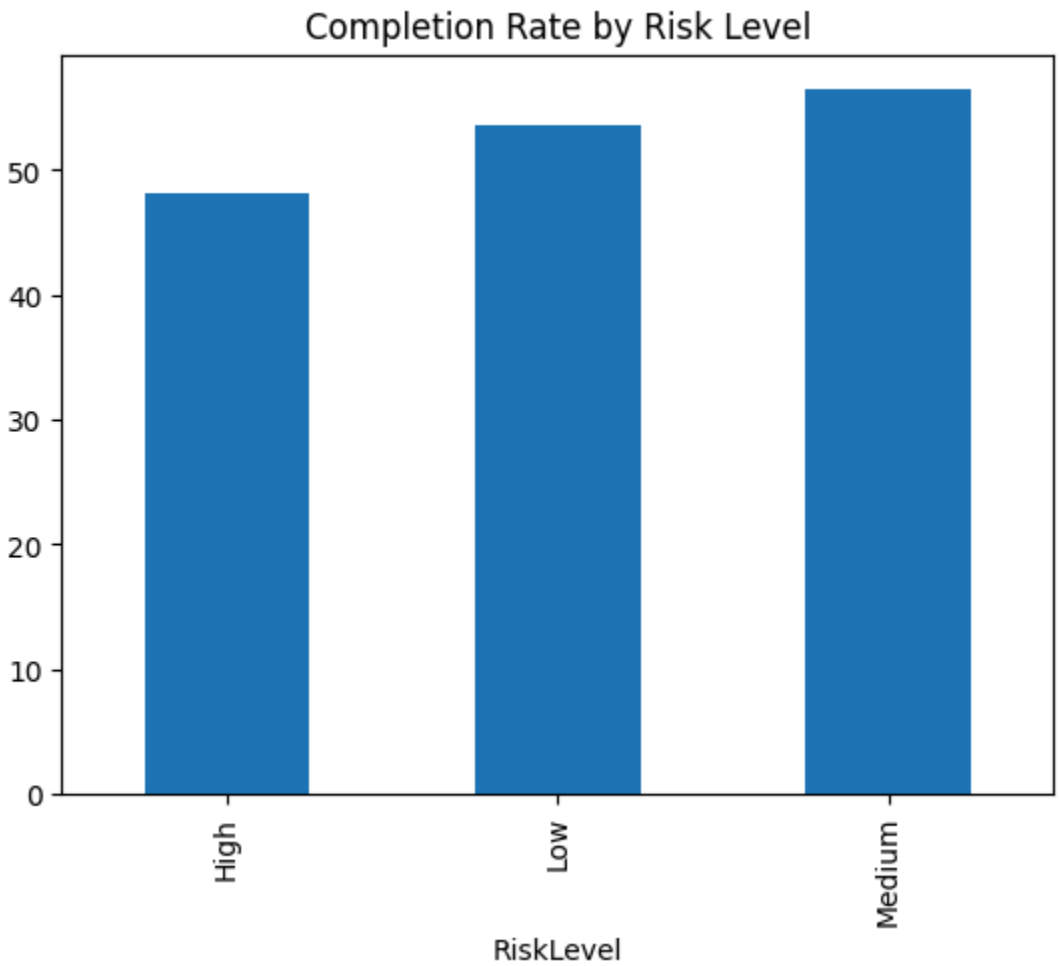
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWa
rning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(**kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 177ms/step
Next forecasted completion rate: [[9.566956]]
```

In [ ]:
```
# Calculate core KPIs
df['CompletionRate'] = df['CompletionPercentage']
df['BudgetVariance'] = df['ActualBudget (EUR)'] - df['PlannedBudget (EUR)']
# df['HoursVariance'] = df['ActualHours'] - df['PlannedHours'] # Columns 'Actu

# Aggregate KPIs
kpi_summary = df[['CompletionRate','BudgetVariance']].mean()
print("KPI Summary:\n", kpi_summary)
```

```
KPI Summary:
 CompletionRate        54.296296
BudgetVariance   -627666.666667
dtype: float64
```

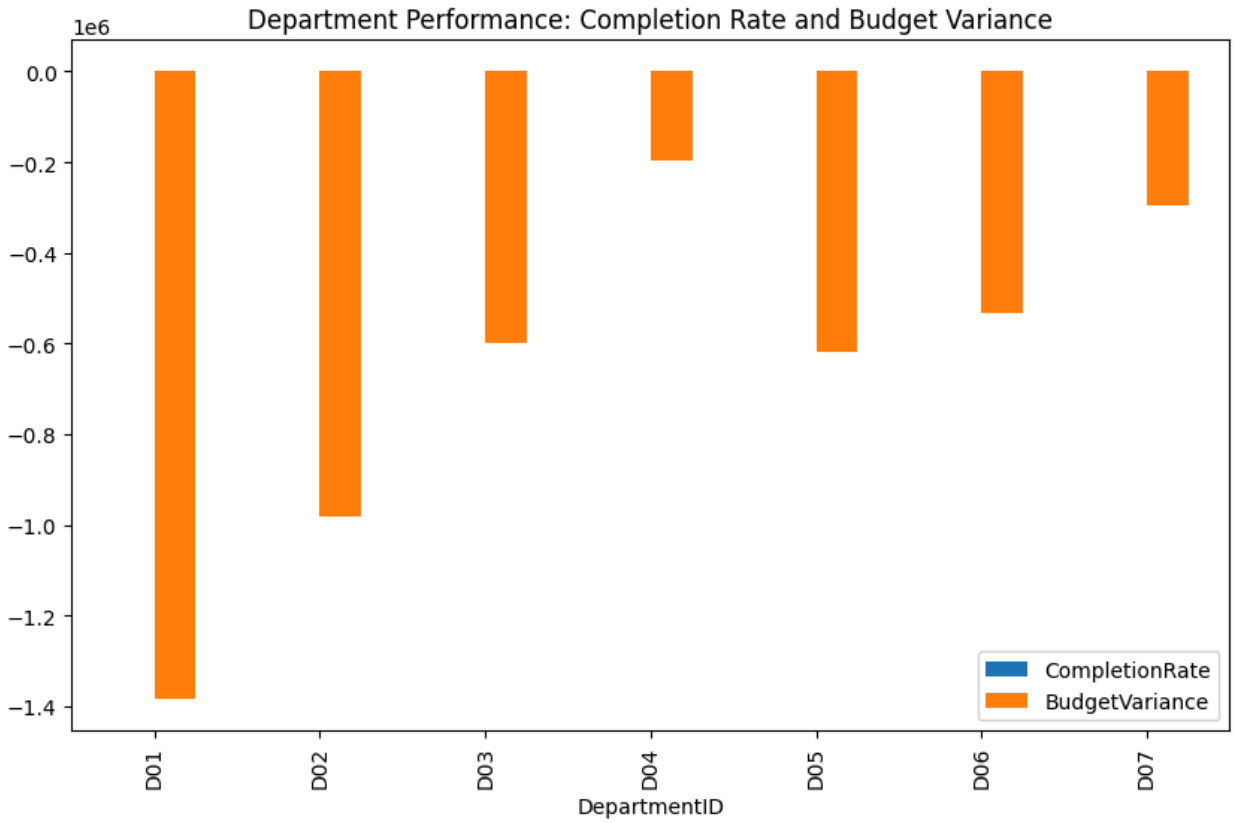
In [ ]:
```
# Completion rate by risk level
risk_completion = df.groupby("RiskLevel")["CompletionRate"].mean()
risk_completion.plot(kind="bar", title="Completion Rate by Risk Level")
```

Out[ ]:
```
<Axes: title={'center': 'Completion Rate by Risk Level'}, xlabel='RiskLevel'>
```



In [ ]:
```
dept_perf = df.groupby("DepartmentID")[["CompletionRate","BudgetVariance"]].me
dept_perf.plot(kind="bar", figsize=(10,6), title="Department Performance: Comp
```

Out[ ]:
```
<Axes: title={'center': 'Department Performance: Completion Rate and Budget V
ariance'}, xlabel='DepartmentID'>
```

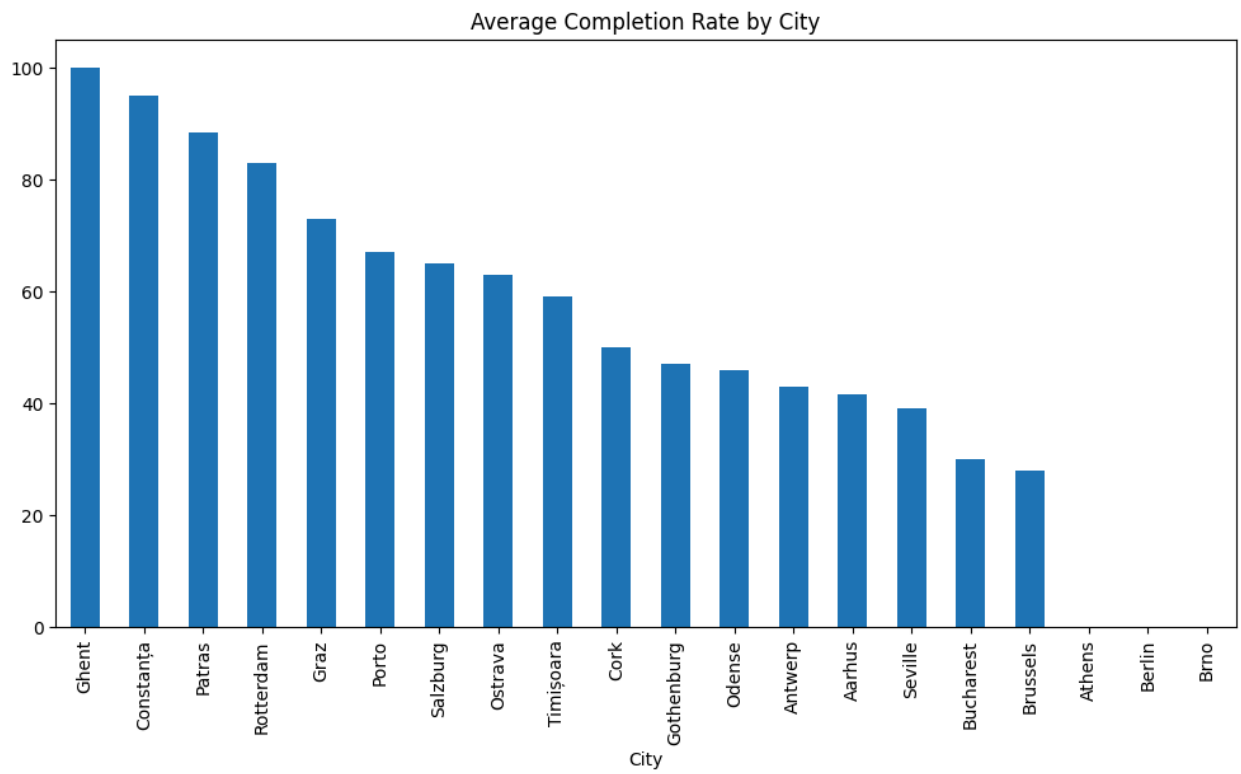


In [3]:

```
# Average completion rate by project country

# Check if 'df' is defined. If not, reload and preprocess it for robustness (e
if 'df' not in locals() and 'df' not in globals():
    import pandas as pd
    import matplotlib.pyplot as plt # Ensure plt is imported for plotting
    file_path = "/content/Fintech_Projects_Challenge32_Dataset.xlsx"
    try:
        df = pd.read_excel(file_path)
        # Reapply essential cleaning steps for 'df'
        df['PlannedStartDate'] = pd.to_datetime(df['PlannedStartDate'], errors
        critical_columns = ['PlannedStartDate', 'CompletionPercentage', 'Actua
        df.dropna(subset=critical_columns, inplace=True)
        df['CompletionRate'] = df['CompletionPercentage']
        df['BudgetVariance'] = df['ActualBudget (EUR)'] - df['PlannedBudget (E
    except FileNotFoundError:
        print(f"Error: Dataset not found at {file_path}. Please ensure the fil
        # Optionally, raise an error or exit if data loading is critical
        raise # Re-raise to stop execution if data cannot be loaded
    except Exception as e:
        print(f"An error occurred during data loading or initial cleaning: {e}
        raise # Re-raise to stop execution on other data errors

# Now proceed with plotting logic, which was the previous modification
if not df['Project_Country'].dropna().empty:
    country_completion = df.groupby("Project_Country")["CompletionRate"].mean(
    country_completion.plot(kind="bar", figsize=(12,6), title="Average Complet
    plt.show()
```

```
else:
    print("No valid data for Project_Country. Attempting to plot by City.")
    if not df['City'].dropna().empty:
        city_completion = df.groupby("City")["CompletionRate"].mean().sort_val
        city_completion.plot(kind="bar", figsize=(12,6), title="Average Comple
        plt.show()
    else:
        print("No valid data for City either. Plotting by RiskLevel as a fallb
        risk_completion = df.groupby("RiskLevel")["CompletionRate"].mean().sor
        risk_completion.plot(kind="bar", figsize=(12,6), title="Average Comple
        plt.show()

# Additional debug info for all columns to understand why they might be empty.
print("\n--- Debugging Data Availability ---")
print("Unique values in Project_Country:", df['Project_Country'].unique())
print("Number of nulls in Project_Country:", df['Project_Country'].isnull().su
print("Number of non-nulls in Project_Country:", df['Project_Country'].notnull

print("\nUnique values in City:", df['City'].unique())
print("Number of nulls in City:", df['City'].isnull().sum())
print("Number of non-nulls in City:", df['City'].notnull().sum())

print("\nUnique values in RiskLevel:", df['RiskLevel'].unique())
print("Number of nulls in RiskLevel:", df['RiskLevel'].isnull().sum())
print("Number of non-nulls in RiskLevel:", df['RiskLevel'].notnull().sum())
```

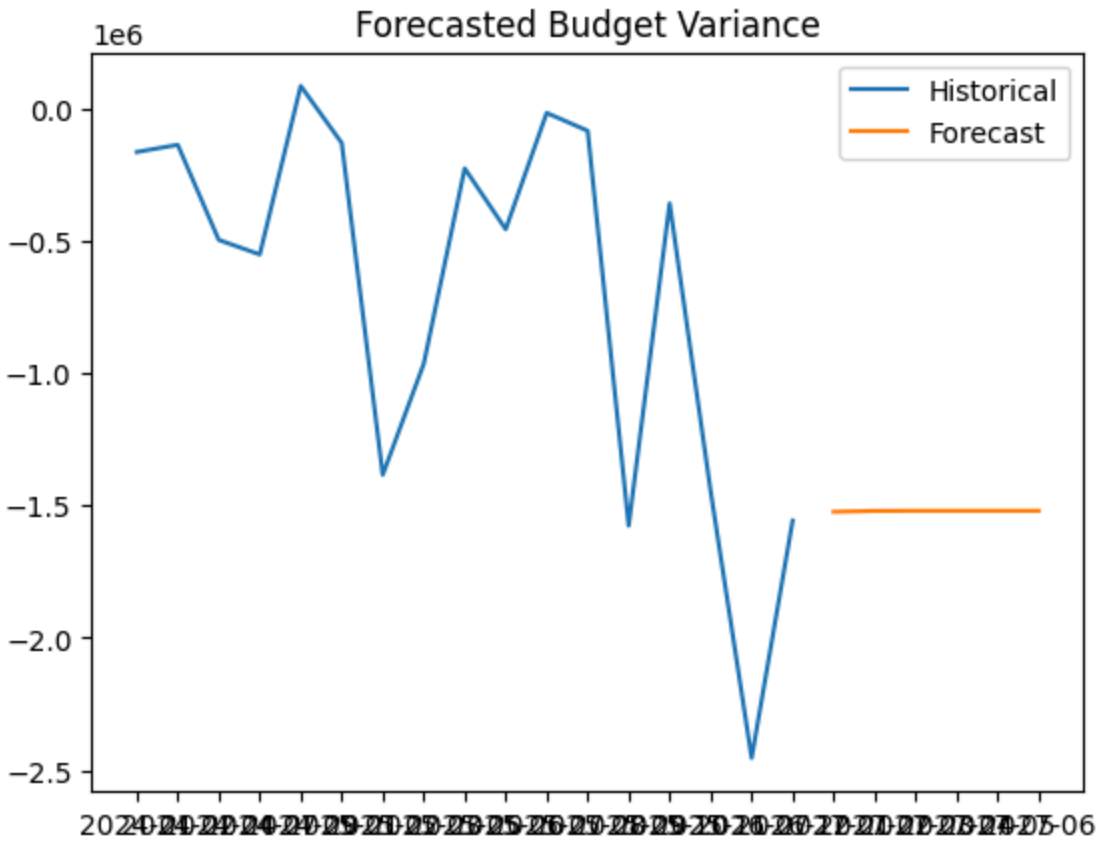
```
No valid data for Project_Country. Attempting to plot by City.
```



```
--- Debugging Data Availability ---
Unique values in Project_Country: [nan]
Number of nulls in Project_Country: 27
Number of non-nulls in Project_Country: 0

Unique values in City: ['Porto' 'Rotterdam' 'Ghent' 'Salzburg' 'Patras' 'Athen
s' 'Brussels'
 'Aarhus' 'Seville' 'Bucharest' 'Odense' 'Antwerp' 'Cork' 'Ostrava'
 'Gothenburg' 'Constanța' 'Timișoara' 'Brno' 'Graz' 'Berlin']
Number of nulls in City: 0
Number of non-nulls in City: 27

Unique values in RiskLevel: ['Medium' 'Low' 'High']
Number of nulls in RiskLevel: 0
Number of non-nulls in RiskLevel: 27
```

In [ ]:

```
# Time-series forecast of budget variance
budget_series = df.groupby("Month")["BudgetVariance"].mean()

model = ARIMA(budget_series, order=(1,1,1))
fit = model.fit()
forecast = fit.forecast(steps=6)

# Adjusting x-axis for plotting PeriodIndex objects
plt.plot(budget_series.index.astype(str), budget_series, label="Historical")
# Generate future PeriodIndex for forecast plotting
future_index = pd.period_range(start=budget_series.index[-1] + 1, periods=6, f
plt.plot(future_index.astype(str), forecast, label="Forecast")
plt.legend()
plt.title("Forecasted Budget Variance")
plt.show()
```



Final Conclusion and Recommendations

Based on the comprehensive analysis of project data, including historical performance and forecasted trends, we can draw several conclusions regarding project management efficiency and financial health. This section summarizes the findings and provides actionable recommendations to optimize future project outcomes.